Synthesising Clustering Algorithms from Benchmark Failure Modes:

Basis Mixtures, Region-Adaptive Pipelines, Learned Routers, a Behaviour-Cloned Agent and Micro-Cluster Graphs

Ashail Maharaj
https://github.com/Ashail33/clustbench

October 2026

Abstract

We report twenty-three rounds of algorithm development carried out inside Clustbench [1], a benchmark harness that re-ranks every candidate against a registry of clustering algorithms on every run. The algorithms were not designed up front; each round started from a failure mode the previous round's benchmark exposed. We contribute (i) a family of exponential-family mixture models whose component log-densities are linear in a shared basis, with random Fourier (FMM), graph Laplacian (LMM) and autoencoder (AMM) bases, plus multi-basis and transform-feature variants; (ii) a physics-inspired method (MRIC); (iii) five targeted one-line fixes of classical methods; (iv) three composed pipelines, each iterated through a v1 → v2 → v3 cycle; (v) a graph-community method, `louvain_knn`; (vi) ten learned algorithm selectors (*routers*); (vii) four machine-proposed algorithms; (viii) `rl_pipeline`, a behaviour-cloned policy that clusters by chaining eight primitive operators; and (ix) graph clustering at linear cost: approximate-graph variants of LMM and spectral clustering, and *micro-cluster graph* methods (MCG) that build the graph on about a thousand k-means micro-clusters linked by competitive Hebbian edges. All results come from a benchmark run with validity controls (five seeds, post-hoc tests, real-data provenance checks, timing of the algorithm call alone, run isolation). Graph-based methods are the strongest single algorithms in-distribution: `lmm_mcg_dual`, LMM, `lmm_mcg` and spectral clustering have mean ARI 0.763–0.769 among 61 algorithms. They cluster by connectivity rather than by distance to a centre, and on manifolds their eigenvectors are cosine modes along the data. The MCG methods keep that accuracy at linear cost. `lmm_mcg_dual` is statistically tied with LMM in-distribution (0.769). On a held-out split the four MCG variants have the four highest means (0.890–0.925 against 0.858 for LMM), though none of the differences survives multiple-comparison correction. At $n = 100{,}000$, `mcg_cf` matches LMM's accuracy 54 times faster. The best routers beat every single algorithm by about 0.05 ARI in-distribution. Retrained without the dataset family under test, they lose 0.03–0.07 and are statistically tied with LMM. On curved shapes they choose centroid methods and score 0.31–0.35, against 0.74–0.86 for graph methods. Targeted fixes frequently traded the diagnosed failure for a new one, and several of our own earlier results did not reproduce.

1 Introduction

No clustering algorithm is best on all data [2, 3], and the classical families fail in characteristic ways. Centroid methods cannot represent non-convex clusters. Mean-based updates are dragged by outliers. Density methods depend on a scale parameter. Graph methods pay quadratic eigendecomposition costs. A companion paper [1] describes Clustbench, the harness used here. It runs a registry of algorithms on a grid of synthetic, real, time-series, graph and adversarial datasets, records quality and cost, and aggregates by average rank and the Friedman test [4].

This paper reports what we built with it. The process was empirical iteration: run the benchmark, name the narrowest failure mode visible in the per-shape and per-outlier breakdowns,

and build the smallest change that addresses it. Section 2 makes this loop explicit. Section 3 describes the algorithms. Section 4 evaluates them in-distribution, on an out-of-distribution split, at $n = 100{,}000$, and on every dataset generator in the harness. It also explains why graph-based methods win on curved data. Section 5 states the lessons, several of them negative.

2 Methodology: empirical iteration

The benchmark's per-algorithm analysis reduced almost every weakness to one of six failure modes: (F1) outlier sensitivity of mean-based updates; (F2) non-convex shape failure of Voronoi-cell methods; (F3) super-linear scaling; (F4) hyperparameter brittleness; (F5) local-minimum search; and (F6) a wrong structural assumption (e.g. subspace clustering on isotropic blobs). Each failure mode has a short menu of known remedies. That observation motivates a five-stage loop:

1. **v1, synthesis.** Compose the mechanisms that already won on each axis: k-means EM for convex blobs, a graph-Laplacian embedding for non-convex shape, posterior weighting for outliers, single-pass trees for scale, data-adaptive parameters for robustness, and diverse ensembles.
2. **Diagnose.** Benchmark v1 and name its *specific* new failure, e.g. "ARI 0.00 on circles because a full-covariance GMM absorbs both rings into one Gaussian."
3. **v2, targeted fix.** Change only the mechanism the diagnosis names.
4. **v3, dispatch.** If v1 and v2 win on different regimes, route between them on a cheap data signature.
5. **Learned dispatch.** Replace hand-coded routing with a model trained on the benchmark's own (task fingerprint, algorithm, ARI) table.

The loop stops when per-iteration lift falls within noise, when the router concentrates on one choice, or when no regime in the grid can show a lift. In that last case the right move is to enlarge the benchmark, not the algorithm.

3 Algorithms

3.1 Exponential-family basis mixtures: FMM, LMM, AMM

GMMs [5] assume Gaussian components. We replace each component with an exponential family whose sufficient statistics are a shared, fixed basis $\phi : \mathbb{R}^d \to \mathbb{R}^M$:

$$p(x) = \sum_{k=1}^{K} \pi_k \, p_k(x; \alpha_k), \qquad p_k(x; \alpha_k) \propto \exp\!\big(\alpha_k^\top \phi(x)\big).$$

The log-partition function is intractable for a general basis. We sidestep it by *self-normalising over the data*:

$$\log p_k(x_i) = \alpha_k^\top \phi(x_i) - \log \sum_{j=1}^{n} \exp\!\big(\alpha_k^\top \phi(x_j)\big),$$

which turns p_k into a discrete distribution over the observed points. The E-step computes responsibilities γ_{ik} as usual. The M-step is concave in each α_k and is solved with one damped Newton step,

$$\big(\mathrm{Cov}_{q_k}[\phi] + \lambda I\big)\, \Delta\alpha_k = \tfrac{1}{N_k} \textstyle\sum_i \gamma_{ik} \phi(x_i) - \mathbb{E}_{q_k}[\phi] - \lambda \alpha_k,$$

where q_k is the softmax of $\alpha_k^\top \phi(x_j)$ over the data. The Hessian is a covariance matrix, so EM converges in a handful of outer iterations, and every iteration is logged as a trajectory step. The three members differ only in ϕ:

- **FMM** uses random Fourier features [6], $\phi(x) = [\cos \omega_m^\top x, \sin \omega_m^\top x]_{m=1}^M$. The frequencies are drawn from a mixture of bandwidths centred on the inverse median pairwise distance, so one model resolves both fine and coarse structure.
- **LMM** uses the bottom eigenvectors ψ of the normalised kNN-graph Laplacian [7], i.e. the spectral embedding, weighted by a per-cluster heat kernel $W_m(\tau_k) = e^{-\tau_k \lambda_m/2}$. Here λ_m is the m-th Laplacian eigenvalue, so τ_k is a learned graph-diffusion bandwidth: small τ_k keeps neighbourhood detail, large τ_k averages over a connected component. LMM can be read as spectral clustering with soft, EM-trained assignments in place of a final k-means.
- **AMM** uses the bottleneck activations of a shallow autoencoder. It is intended for high-dimensional sparse inputs, where kNN distances lose meaning.

Relation to prior work.

- **The name "Laplacian mixture model" is taken.** Korenblum [8] combines Laplacian eigenspaces with finite mixture modelling to obtain fuzzy partitions. That model is fitted by globally optimising a linear transformation of the eigenvectors (a linearly constrained concave quadratic programme), with optimal-recovery guarantees for a class of cluster graphs. Our LMM shares the Laplacian-eigenvector basis and soft assignments. It differs in fitting per-cluster exponential-family log-densities on that basis by EM, with self-normalisation over the data and a learned per-cluster heat-kernel diffusion time. We keep the abbreviation for continuity with the code, but the method should be read as a variant within that line of work, not a new model class.
- **FMM's ingredients are known.** EM for mixtures of exponential families is classical [9]. Exponential families over kernel or random-feature bases are studied as density estimators [10], typically fitted by score matching rather than by maximum likelihood, because of the partition function. Random Fourier features have been used for kernel clustering [11]. We did not find this exact combination: a mixture whose components are random-Fourier exponential families with the log-partition self-normalised over the sample, fitted by Newton-step EM. Our search was limited, though, and we make no strong novelty claim.

3.2 Combined bases and transform features (round 17)

Two questions followed from the basis mixtures: does combining bases help, and can classical integral transforms replace them?

- **TBMM** concatenates the graph-Laplacian, random-Fourier and autoencoder bases into one ϕ, warm-started from a spectral solution. `tbmm_fl` drops the autoencoder block.
- **FLHMM** clusters on three per-point transform features:
  - random Fourier features (an empirical characteristic function);
  - a Laplace transform of each point's distance distribution, $\frac{1}{n}\sum_j e^{-s\,d_{ij}}$ on a log-grid of s;
  - the envelope and instantaneous phase of the analytic (Hilbert) signal of the density projected on random directions.

  The blocks are scaled to equal variance and concatenated, with a mixture head (`flhmm`) or a k-means head (`flhmm_km`).
- **FLH-cycle** keeps one mixture per transform view and shares only the soft assignments. Each step runs one view's M-step, then an E-step, cycling Fourier → Laplace → Hilbert. This is co-EM [12, 13] with transforms as views. `flh_cycle_product` instead computes each E-step from all views' likelihoods (block-coordinate EM on the product of experts), with a 50% hand-off.

3.3 MRIC: magnetic-resonance-inspired clustering

MRI separates tissues because nuclei in different chemical environments relax at different rates. MRIC treats each point as a spin and its kNN neighbourhood as its environment. The first

principal axis plays the static field B_0. Local density (inverse mean kNN distance) and local anisotropy (from the eigenvalues of the local covariance) map to per-point T_1/T_2 relaxation times. A simulated free-induction-decay signal is sampled into a per-point signature, and k-means clusters the signatures.

3.4 Targeted fixes of classical methods

Each fix changes one mechanism named by the diagnosis:

- `dbscan_auto`: ε set from the knee of the k-distance plot [14] (F4).
- `meanshift_robust`: bandwidth estimated on a trimmed sample (F1/F4).
- `clarans_pp`: k-means++ medoid seeding [15] (F5).
- `pwcc_diverse`: PWCC [16] with heterogeneous bases $\{k\text{-means}, \text{spectral}, \text{GMM}\}$ in place of three k-means variants (F2).
- `kmeans_trimmed`: drop the farthest 10% of members from each centroid update (F1).

3.5 Composed pipelines and their v1 → v3 lineages

RAPID (Region-Adaptive Partitioning with Iterative Density). *v1* partitions the data into density-connected regions with auto-ε DBSCAN. It computes a convexity signature per region and sends convex regions to k-means++ and non-convex regions to spectral clustering, then reassigns DBSCAN noise by 1-NN. Diagnosis: the k-distance knee is contaminated by the outliers it should ignore (outlier ΔARI -0.145). *v2* adds a stage-0 LOF [17] pre-filter that holds out the top 10% of LOF scores. *v3* runs the v2 path only when the estimated LOF outlier fraction exceeds 5%.

AURA (Adaptive Unified Robust Algorithm). *v1* computes a Nyström-approximated [18, 19] normalised-Laplacian embedding Y and fits a full-covariance GMM on it, aiming for spectral shape-handling plus GMM outlier robustness. Diagnosis: ARI 0.00 on circles, because the GMM merges both rings into one elongated component and the eigen-columns differ in scale by $\sim 20\times$. *v2* z-scores Y column-wise and runs k-means. *v3* counts the informative embedding axes, $r = |\{j : \mathrm{sd}(Y_{:j}) \geq 0.1 \max_{j'} \mathrm{sd}(Y_{:j'})\}|$, and routes $r \geq 2$ to the v2 path (z-scored k-means) and $r < 2$ to the v1 path (raw GMM). Moons have one informative axis and circles two, so each version runs on its own regime.

META-CLUSTERER. *v1* computes four fingerprints (PCA effective dimension, Laplacian eigengap, LOF outlier fraction, kNN density skew) and dispatches with hand-written rules. Diagnosis: the eigengap rule never fires on clean circles, whose gap is *large*. *v2* replaces the eigengap with a convexity ratio of the k-means partition and the kNN-graph modularity [20] of that partition. It also adds a silhouette probe that fits three candidates on a 20% subsample when a rule fires with low margin. *v3* runs both routers and probes only when they disagree.

3.6 `louvain_knn`: a graph-native method for a coverage gap

Enlarging the benchmark to graph-node features exposed Zachary's karate club [21] as a regime where every algorithm scored below 0.43 ARI. With 34 nodes and 16 features, re-embedding the features drowns the Fiedler signal. `louvain_knn` builds a symmetrised kNN graph on the features and optimises modularity directly with Louvain [22]. It then reconciles the discovered community count with the requested k: it merges the smallest community into its most densely connected neighbour, or splits the largest community by spectral bipartition. It falls back to a self-contained NumPy Louvain implementation, and then to spectral clustering, if dependencies are missing.

Table 1: Learned router generations.

version	mechanism
v1	7 features ($\log n$, d, k, PCA effective dimension, convexity CV, LOF outlier fraction, density skew); K=5 nearest training tasks; pick the lowest mean ARI rank.
v2	15 features: v1 plus 3-iteration k-means++ probe statistics (inertia ratio, silhouette, size CV, centroid-shift ratio), Levina–Bickel intrinsic dimension [23], multi-scale kNN distances.
v3	17 features: v2 plus an auto-ε DBSCAN probe (clusters found, noise fraction); one distance-weighted kNN *regressor per algorithm* predicting ARI; dispatch on the argmax.
v4	20 features (adds real-data, image-domain and outlier-extremity flags); gradient-boosted regressors per algorithm; fall back to v3 if the two disagree by > 0.05 ARI.
v5	Stacker choosing between v3 and v4 per task [24].
v6	Probe router: run candidates on a subsample and dispatch on silhouette.
v6b	v6 with a multi-metric probe (silhouette, Calinski–Harabasz, Davies–Bouldin, k-neighbour purity).
v6c	v6b with an inverse-cost term at 20% weight.
v7	Random-forest stacker over {v3, v6}, trained to predict where v6 beats v3 by > 0.02.
v8	11-feature enriched fingerprint (distance kurtosis, mass Gini, kNN density variance, covariance effective rank), out-of-distribution-triggered top-3 ensemble, aggregation by mean ARI.

3.7 Learned routers

A router computes a fingerprint of the input, chooses an algorithm from the registry, and runs it. It is trained on the benchmark's own results: every historical task is regenerated from its specification, fingerprinted, and joined with every algorithm's ARI on it. Training happens at import time, so a router retrains whenever the results file changes. Table 1 summarises the ten generations.

All routers exclude themselves and their siblings from the candidate set to prevent recursion. For honest in-distribution evaluation they drop an exact fingerprint self-match (task-level leave-one-out).

3.8 Machine-proposed algorithms

Card mutator. Clustbench encodes every algorithm as a card listing its mechanisms, inductive biases and complexity [1]. A mutator searches card space with swap, add-bias, crossover and stack operators and scores candidates by a bias-matching ARI upper bound. Its top proposal, `mutant_kmeans_meta`, crosses k-means with META-CLUSTERER-v2. It keeps the k-means partition when a convexity probe and a silhouette check both pass, and otherwise hands off to the META-v2 router.

Multi-agent discovery (round 14). LLM agents running in parallel proposed algorithms, ordinary datasets and adversarial datasets. Separate agents implemented, integrated and benchmarked them. Three algorithms came out of this round:

- `density_peaks_bridge`: density peaks [25] plus one round of normalised-Laplacian label spreading [26], which reassigns borderline halo points and keeps true outliers.
- `bico_stream`: a BICO-style [27] streaming coreset with merge-and-reduce [28], followed by weighted k-means++ on the coreset.

- `isomap_bgmm`: an Isomap [29] geodesic embedding followed by a Dirichlet-process Bayesian GMM [30].

Trident (round 15). Designed from the round-14 winners table, Trident runs one cheap probe per winning family (centroid: k-means; mixture: diagonal GMM; graph: `louvain_knn`). It gates on silhouette and pairwise-ARI agreement between the probes, and escalates to `isomap_bgmm` on a heavy-tail signature or to AURA-v3 on density variation. Selection is built into the algorithm rather than delegated to an external router.

3.9 `rl_pipeline`: clustering as sequential decision-making

Clustbench's trajectory layer records iterative algorithms as state–action–cost sequences [1]. We ask whether a policy can *compose* an algorithm from primitives. The environment has eight actions drawn from the registry's dominant mechanisms: `kpp_init`, `assign_to_centers`, `update_centers` (k-means); `medoid_swap` (CLARANS); `ward_merge` (Ward); `density_partition` (DBSCAN); `eigen_embed` (spectral); and `outlier_trim`. The state is a 20-dimensional vector: n, d, k, cost, silhouette, plateau signals, outlier mask statistics and centroid statistics. The reward is $\mathrm{clip}(-\Delta c_t/c_0)$.

Hand-written action sequences reproduce their source algorithms: a k-means sequence reaches ARI 1.0 on blobs, and the spectral sequence `kpp`→`eigen_embed`→`assign`→`update` reaches 1.0 on circles. For behaviour cloning [31], we drove k-means, CLARANS, Ward, spectral clustering and DBSCAN through the environment, which produced 50 episodes and 419 (state, action, reward) rows. We trained an MLP policy (cross-entropy, with count$^{-1/4}$ class re-weighting) and an MLP value head (discounted return, γ=0.95) on an 80/20 episode split. At inference the agent runs 16 stochastic rollouts, and a tiered selector keeps the best non-embedding rollout with silhouette ≥ 0.40, or else the best embedding rollout.

3.10 Graph clustering without the point graph (rounds 19–23)

LMM and spectral clustering spend almost all their time on the kNN graph over all n points. Building it exactly costs $O(n^2 d)$: 8.7 s of LMM's 11.2 s at $n = 50{,}000$, $d = 20$. Its eigenvectors then cost $O(n \cdot \text{iterations})$, and on manifold-shaped data an unpreconditioned LOBPCG needs 184 iterations (16.6 s at $n = 10^5$). We attacked both costs.

- **Approximate graph, preconditioned eigensolver** (`lmm_fast`, `spectral_fast`). The graph is built with NN-descent [32] once $n \geq 10^4$ and $d > 10$. It is 12× faster than exact search at $n = 2 \cdot 10^5$ and recovers 95–98% of the true neighbours at $d = 20$. LOBPCG [33] is preconditioned with plain-aggregation algebraic multigrid (PyAMG [34]), which needs about six iterations on the same data.
- **MCG: a graph on micro-clusters** (`mcg`). Fit mini-batch k-means with $m = \min(1000, \max(n/10, 10k), n/3)$ centres on a subsample: about ten points per micro-cluster, at least ten per cluster, and never fewer than three points each (round 24). Without the last cap, small data with many clusters got one point per micro-cluster: 400 for Olivetti's 400 faces, where `lmm_mcg` scored 0.02. The cap raises that to 0.37 and leaves larger data unchanged. Each point votes for an edge between its nearest and second-nearest centre: competitive Hebbian learning [35], whose edge counts are the CONN similarity of Taşdemir and Merényi [36]. Edges therefore exist only where data lies *between* two centres. Counts are weighted by $\exp(-\|c_i - c_j\|^2/2\sigma^2)$, and spectral clustering runs on the $m \times m$ graph; each point takes its centre's label. The cost is $O(nmd)$ plus a fixed $O(m^3)$. Spectral clustering on k-means centroids is KASP [37], which links centroids by their positions alone. Spectral clustering on a kNN graph of the same centroids scores 0.27 on rings at $n = 2 \cdot 10^4$, where the point-voted edges score 0.97.

- **Tiers and CF triples** (`mcg_cf`). Micro-clusters are built coarse-to-fine: a k-means with about $2\sqrt{m}$ centres, then local splits in proportion to each coarse cluster's size, as in BIRCH's CF tree [38]. Each edge's bandwidth then comes from the two micro-clusters' own spread, read from their clustering-feature triples $(N, \mathrm{LS}, \mathrm{SS})$ as $R^2 = \mathrm{SS}/N - \|\mathrm{LS}/N\|^2$:

$$w_{ij} = \mathrm{count}_{ij}\, \exp(-\|c_i - c_j\|^2/\gamma(R_i^2 + R_j^2)),$$

  a locally scaled affinity in the sense of self-tuning spectral clustering [39]. The parts fix opposite failures:
  - tiers alone fix small real data (breast cancer 0.24 → 0.45) but break extreme outliers (0.56 → 0.27);
  - CF weights alone fix outliers (0.81) but not small data.

  Multi-resolution anchor graphs [40, 41] are the closest prior work.
- **LMM on the micro-cluster basis** (`lmm_mcg`). The bottom eigenvectors of the micro-cluster graph replace LMM's point-graph basis. Each point takes its nearest micro-cluster's row, and LMM's EM then runs on the points.
- **A coarse-grained Laplacian for manifolds** (`lmm_mcg_dual`). The Gaussian edge weights that sharpen cuts across gaps also bend the geometry of smooth, gap-free sheets (§4.4). With plain edge counts (a boundary-flux estimate) and the mass-weighted eigenproblem $(D - W)v = \lambda\, \mathrm{diag}(N)\, v$, the micro-cluster Laplacian is a finite-volume coarse-graining of the point graph's. It recovers LMM's eigenvectors. Each point's basis is then blended linearly between its two nearest micro-clusters. `lmm_mcg_dual` gives EM both bases side by side ($2k$ columns).

We also tried two other second stages on the same micro-cluster graph, and both lost:

- SVD of the soft point-to-micro-cluster bipartite graph [40]: 0.59–0.70 in-distribution;
- support vector clustering [42] on the count-weighted micro-clusters, with the kernel width bisected to give k components: 0.66–0.69.

The first blurs boundaries between touching clusters. The second cannot separate clusters joined by a bridge.

4 Results

Unless stated otherwise, numbers come from the controlled benchmark rerun [1]:

- an in-distribution split of 61 algorithms on 140 tasks (28 task variants from 20 dataset families, seeds 1–5, $n \leq 1000$);
- a leave-one-dataset-family-out (LOFO) evaluation of the routers on the same tasks;
- an out-of-distribution (OOD) split of 62 algorithms on 35 tasks with unseen (n, d, k), outlier extremities and seeds 100–104;
- a large-n sweep up to $n = 50{,}000$, a big-data sweep up to $n = 200{,}000$, and graph clustering at $n = 100{,}000$ on eleven families;
- an every-dataset run of 63 algorithms on 180 tasks from all 36 dataset generators.

The MCG settings (number of micro-clusters, edge weighting, γ, hard assignment) were chosen on the OOD split and a 20k-point development suite, with the in-distribution tasks held out. The `lmm_mcg_dual` basis was developed on the curved shapes of the every-dataset run, so its numbers there are not held out. Only `fit_predict` is timed (median of three runs, one BLAS thread). Lineage tables (Tables 2–3) come from the round in which each version was introduced, on the 16-task demonstration grid and before the controls, and are marked as such.

4.1 Targeted fixes

Table 2 reports the five fixes. Auto-ε is the single largest intervention in the project (+0.589): it takes DBSCAN from one cluster on every task to ARI 0.83 on circles. Three fixes exposed second-order failures.

Table 2: Targeted fixes (16-task demonstration grid, round 2).

fix	ARI before	ARI after	ΔARI	wall time (s)
`dbscan` → `dbscan_auto`	0.000	0.589	+0.589	0.020 → 0.035
`meanshift` → `meanshift_robust`	0.470	0.622	+0.151	0.229 → 0.288
`clarans` → `clarans_pp`	0.581	0.656	+0.075	0.018 → 0.017
`pwcc` → `pwcc_diverse`	0.702	0.751	+0.049	0.145 → 0.223
`kmeans` → `kmeans_trimmed`	0.701	0.703	+0.003	0.024 → 0.024

Table 3: v1/v2/v3 lineages (16-task demonstration grid, rounds 3–4). Per-shape columns are mean ARI. Rank is out of 31.

pipeline	version	mean ARI	mdcgen	aniso.	moons	circles	rank
AURA	v1	0.733	0.87	1.00	0.37	0.00	9
	v2	0.764	0.77	1.00	0.01	1.00	–
	v3	**0.875**	0.86	1.00	0.55	1.00	**2**
META	v1	0.774	0.90	1.00	0.42	0.16	–
	v2	**0.879**	0.90	1.00	0.42	1.00	4
	v3	0.827	0.90	1.00	0.42	0.58	8
RAPID	v1	**0.843**	0.76	1.00	0.85	0.84	5
	v2	0.774	0.73	0.91	0.85	0.57	–
	v3	0.843	0.77	1.00	0.85	0.84	7

- `kmeans_trimmed` barely moves, because a 10% trim cannot absorb a 20% outlier rate.
- `clarans_pp` is *more* outlier-sensitive than CLARANS, because k-means++ samples proportionally to squared distance and so prefers outliers as seeds.
- `dbscan_auto`'s knee is itself inflated by outliers (outlier ΔARI −0.475).

4.2 Pipeline lineages

Table 3 follows each pipeline through v1, v2 and v3.

- **AURA** is the clean success of meta-dispatch. v2 traded moons (0.37 → 0.01) for circles (0.00 → 1.00), because z-scoring amplifies noise in the near-empty eigen-directions of moons. v3's effective-rank switch recovered both (moons 0.55, circles 1.00) and climbed from rank 9 to rank 2 of 31, with +0.142 ARI over v1.
- **META-CLUSTERER** v2 was a strict improvement: replacing a brittle rule fixed circles (0.16 → 1.00) and broke nothing. v3's probe then *lowered* the score by overruling correct v2 routes, because subsample silhouette does not track ARI on non-convex data.
- **RAPID** v2 closed most of the outlier gap (−0.145 → −0.050) but regressed on shape, because LOF removes real points when cluster density is comparable to outlier density. v2 never beat v1 on any regime, so v3's gate always selected v1 and v3 tied it.

RAPID v1 was, at the time, the only algorithm in the registry strong on both moons (0.85) and circles (0.84).

4.3 Final standings

Table 4 places every new algorithm among all 61 (62 OOD), and Figure 1 breaks the results down by dataset family. Configuration-level 95% bootstrap intervals are about ±0.09 ARI, and a Nemenyi critical difference cannot separate the top fifteen algorithms in-distribution. Read adjacent ranks as ties unless a pairwise test says otherwise [1]. Paired tests below are Wilcoxon signed-rank tests with Holm correction [43]. For the MCG family the correction covers twelve comparisons: four variants against LMM, spectral clustering and router v3.

- **Graph methods lead among single algorithms, and the MCG family ties them at a fraction of the cost.** `lmm_mcg_dual` (0.769), LMM (0.765), `lmm_mcg` (0.764) and spectral clustering (0.763) rank 8th–11th, behind only the routers. No MCG variant differs from LMM or spectral in-distribution (Holm $p \geq 0.42$). OOD, the four MCG variants have the four highest means (`lmm_mcg` 0.925, `mcg_cf` 0.917, `lmm_mcg_dual` 0.892, `mcg` 0.890), ahead of TBMM (0.869), spectral (0.860) and LMM (0.858). None of those gaps survives the correction (smallest Holm $p = 0.12$). By average rank, flat `mcg` leads the OOD split.
- **LMM remains the best full-graph algorithm we built.** It solves circles (1.00), its inlier-only ARI on the 15 contaminated tasks is 0.990, and on the same graph Laplacian it scales far better than spectral clustering: a log–log slope of 1.5 against 2.9, and 6 s against 297 s at n=30,000.
- **Combining bases neither helps nor hurts; transforms hurt.** TBMM ties LMM in-distribution (0.760, Holm $p = 0.36$) and is strong OOD (0.869), and `tbmm_fl` matches it 5× faster: the Laplacian block carries the signal. FLHMM's transform features are significantly worse than LMM (−0.113), with one genuine win: real ECG200 (0.356, the best of any single algorithm; LMM 0.196). Literal co-EM cycling hurts (−0.211 against its own k-means start). Computing E-steps from all views helps (+0.034) but stays far below LMM. AMM ranks 44th.
- **`louvain_knn` closed the gap it was built for.** It is the best non-router on the karate graph (0.73), moons (1.00) and Digits (0.55), and keeps moons at 1.00 up to n=50,000. It is weak on imbalanced blobs (0.57) and power-law Student-t mixtures (0.50).
- **FMM, MRIC and the round-14 proposals are mid-pack or lower.** Density peaks with a bridge won its own mini-grid but ranks 51st of 61.

4.4 Why graph methods win on curved data

k-means is good on blobs because it assigns points by straight-line distance. LMM and spectral clustering effectively assign points by distance *along the data*. The kNN graph only links nearby points, and the graph Laplacian's bottom eigenvectors give coordinates in which straight-line distance approximates diffusion distance on the graph [44, 45]. Clustering in those coordinates groups points by connectivity.

The S-curve and Swiss roll make this concrete. Their true clusters are equal-mass slices of the curve parameter t, with no gaps between slices. On both, LMM's first three non-trivial eigenvectors are the cosine modes $\cos(m\pi s)$ of position s along the curve:

- $|r| = 1.00, 1.00, 0.99$ on the S-curve;
- $|r| = 0.99, 0.97, 0.94$ on the Swiss roll;
- correlation with the sheet's height below 0.05 on both.

The embedding unrolls the sheet (Figure 2, left). Graph distance tracks distance along the curve with rank correlation 0.97–0.99. Straight-line distance tracks it with 0.88 on the gently bent S-curve but only 0.25 on the Swiss roll, whose layers are close in 3-D. So k-means scores 0.91 on the S-curve but 0.21 on the Swiss roll, while LMM scores 0.80 and 0.73. Clustering on the true t itself scores 0.84 (the truth slices by mass, k-means on t by width), so LMM is near the ceiling.

The same analysis explains a weakness of `lmm_mcg`. Its Gaussian edge weights distort these modes on the Swiss roll ($|r| = 0.89, 0.62, 0.22$; Figure 2, middle). The coarse-grained count Laplacian of §3.10 recovers them ($|r| = 1.00, 0.97, 0.92$; right) at the micro-cluster cost. That basis gives 0.82 and 0.73 on the S-curve and Swiss roll, level with LMM, but it cuts gaps less sharply: 0.863 OOD against 0.925 for the CF basis. The dual basis keeps most of both.

4.5 Graph accuracy at linear cost

Figure 3 places every graph method at $n = 100{,}000$, on eleven families at $d = 20$ with two seeds.

- LMM scores 0.837 in 40 s; exact spectral clustering is capped at $2 \cdot 10^4$.

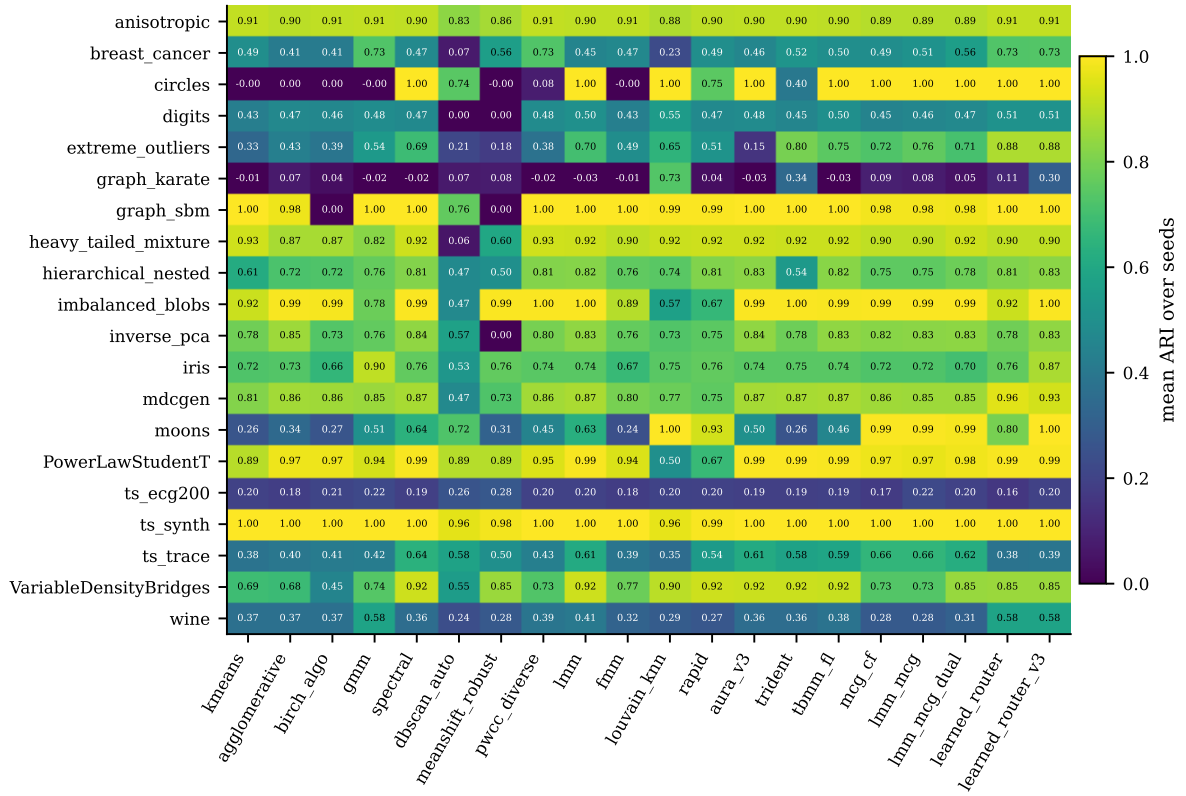


Figure 1: Mean ARI by dataset family (in-distribution) for classical baselines and the new algorithms. `louvain_knn` is the only non-router to solve moons and the karate graph together; the MCG methods reach 0.98–0.99 on moons; LMM, spectral clustering, AURA-v3 and the MCG methods solve circles; GMM is strongest on Iris and Wine; the real UCR time series (ECG200, Trace) are hard for every algorithm.

- The approximate-graph variants keep the accuracy at a fifth of the time: `lmm_fast` 0.844 in 8.5 s and `spectral_fast` 0.832 in 8.9 s. Without the AMG preconditioner they had scored 0.836 and 0.776, because plain LOBPCG did not converge on rings.
- The MCG family is fastest and as accurate: `mcg_cf` 0.859 in 0.74 s (54× faster than LMM), `mcg` 0.853 in 1.5 s, `lmm_mcg` 0.858 in 2.0 s and `lmm_mcg_dual` 0.858 in 3.3 s.
- Centroid methods are faster still but score about 0.56 on these families.

On the big-data axes `mcg_cf`'s wall time grows with exponents 0.32, 0.23 and 0.13 in n, d and k. It finishes $n = 2 \cdot 10^5$ in 1.1 s and the $10^5 \times 100 \times 50$ corner in 1.3 s, both at ARI 1.00; LMM needs 102 s for the corner. The EM-based variants inherit LMM's Newton step, which allocates a (k, M, n) array for a basis of size M: `lmm_mcg` takes 21 s at $k = 50$, and `lmm_mcg_dual` ($M = 2k$) runs out of memory there, as LMM does at $k = 200$.

4.6 Every algorithm on every dataset

The every-dataset run [1] covers all 36 generators at $n = 2000$, both in their native low dimension and padded to $d = 10$, plus every real dataset at close to native size. Figure 4 shows the 2-D shapes.

- **On curved shapes, every top method is a graph method.** Averaged over moons, circles, rings, spirals, the S-curve and the Swiss roll, the leaders are `mcg` (0.863), `lmm_mcg_dual` (0.833), `lmm_mcg` (0.812), `mcg_cf` (0.803), spectral clustering (0.776) and LMM (0.738). Centroid methods score 0.21–0.26 and routers 0.31–0.35.
- **Flat MCG is best in two dimensions.** On 2-D moons, 5 rings and 5 spirals it scores 0.99,

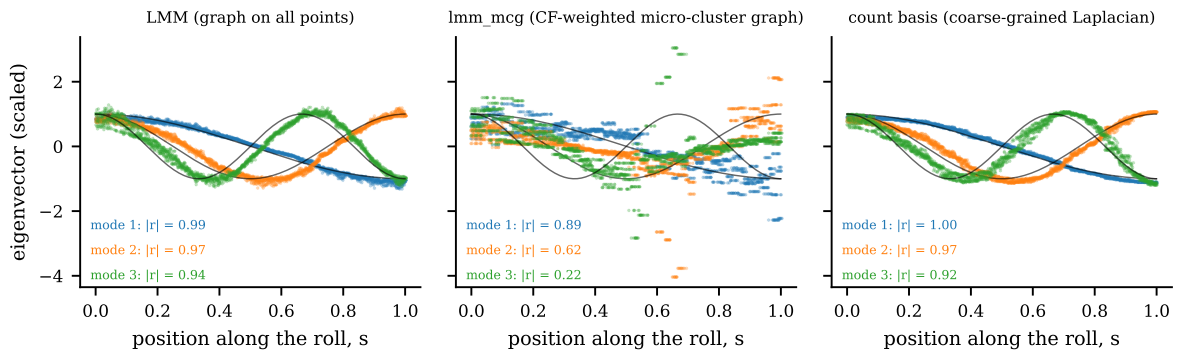


Figure 2: Swiss roll (n=2000): the first three non-trivial eigenvectors of three bases against position s along the roll (dots), with the cosine modes $\cos(m\pi s)$ (black lines). Left: LMM's graph on all points. Middle: the CF-weighted micro-cluster graph of `lmm_mcg`. Right: the coarse-grained count Laplacian of `lmm_mcg_dual`.

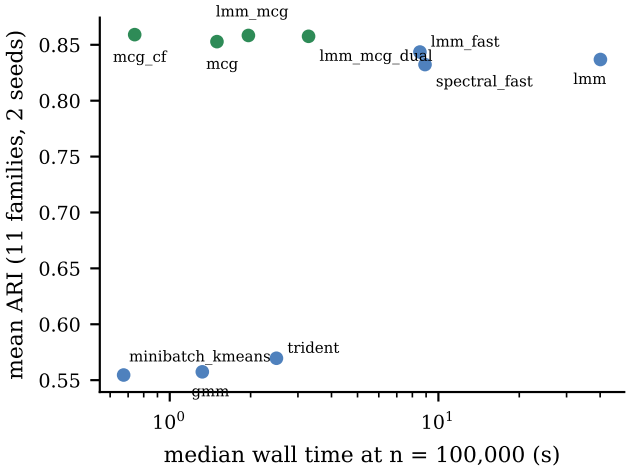


Figure 3: Graph clustering at $n = 100{,}000$ ($d = 20$, eleven families, two seeds): mean ARI against median wall time. Green: micro-cluster graph methods; blue: other algorithms.

0.90 and 0.76. The tiered variants score about 0.6 on all three, and spectral clustering scores 0.25 on the spirals. At $d = 10$ the tiered variants catch up. A plausible but untested cause is that, in 2-D, neighbouring arms are closer than a micro-cluster's CF spread.

- **Mixtures favour the full-graph EM models.** TBMM, LMM and spectral clustering lead at 0.78–0.79 and MCG trails (0.67–0.76). On real data the MCG variants score 0.35–0.38, against 0.36 for LMM and 0.49 for the best router. On Olivetti faces they reach 0.37–0.40, level with LMM (0.40), now that the micro-cluster count is capped at $n/3$.

Over all 180 tasks the four MCG variants have the highest means: `lmm_mcg_dual` 0.686, `lmm_mcg` 0.680, `mcg_cf` 0.674 and `mcg` 0.667. `lmm_fast` also scores 0.667, spectral clustering 0.665 and LMM 0.659. The best router ranks 16th (0.580).

4.7 Routers: in-distribution gains that do not transfer

Figure 5 plots every router generation on three evaluations.

- **In-distribution, routers hold the top seven ranks.** The best beats LMM by 0.051 ARI (paired Wilcoxon, Holm $p = 2 \times 10^{-4}$).
- **Within the lineage, v3 beats v1** (Holm $p = 0.003$). v2's probe features and v3's switch

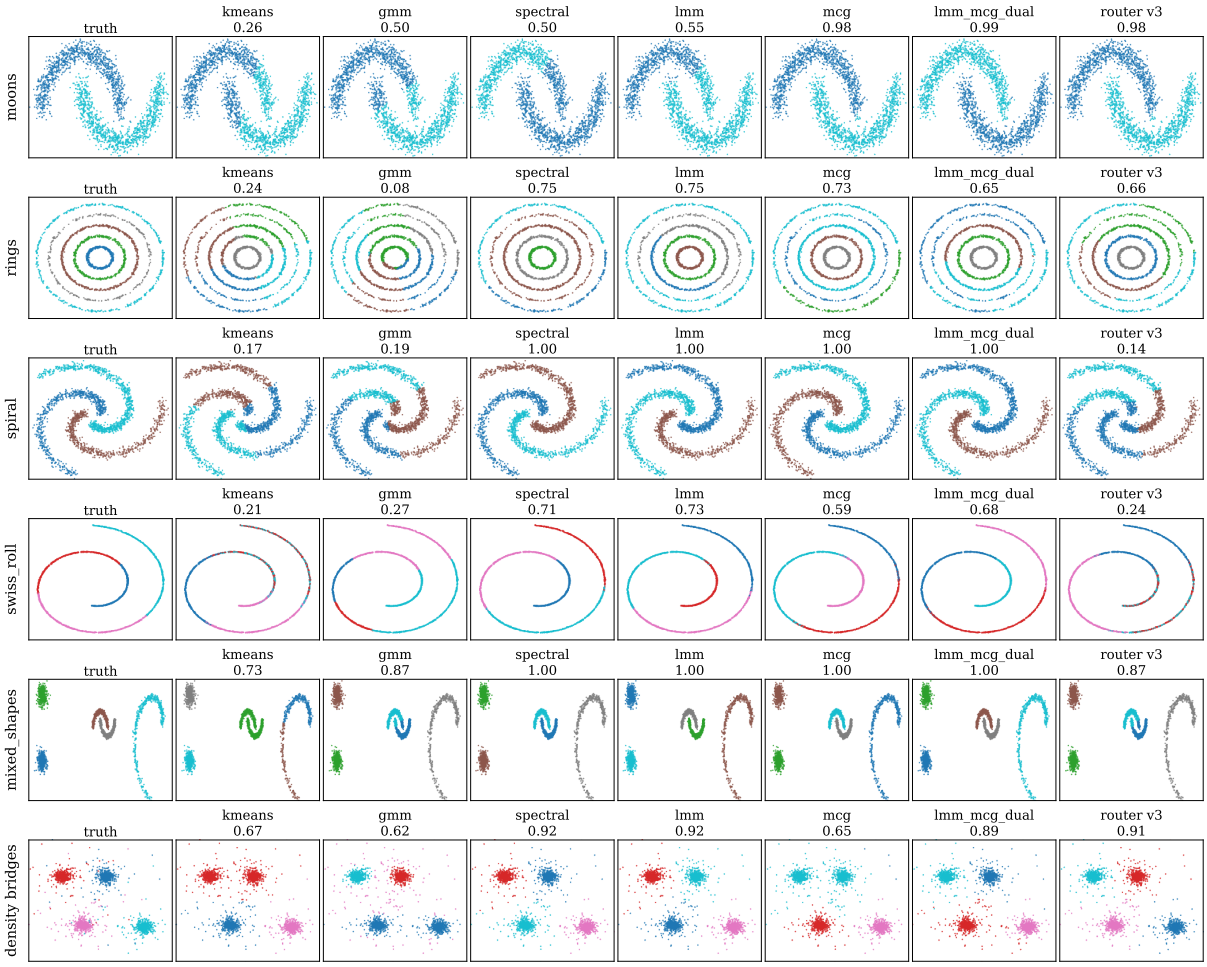


Figure 4: Every-dataset run, seed 1: six 2-D shapes coloured by the ground truth and by seven algorithms, with ARI above each panel (the Swiss roll is shown in its unrolled x–z view).

from rank voting to per-algorithm ARI regression each added roughly 0.02–0.03 ARI when introduced.

- **Architectural changes helped only when they added a new signal.** v4 used the same features with a different model and overrode v3 on 51 of 52 tasks without improving, so the v5 stacker over {v3, v4} degenerated to always-v3. v6's probe signal is different in kind and won exactly one task v3 missed (inverse-PCA, 0.00 → 0.38). That was enough for the v7 stacker over {v3, v6} to take rank 1 in round 8.
- **Silhouette is a biased probe.** As a probe metric it favours compact wrong answers on non-convex data, and adding a 20% cost term (v6c) dethroned spectral clustering on close calls where it was correct.

Leave one family out. Retraining each router without the family under test removes the leakage of sibling seeds.

- v3, v5 and v7 fall from 0.815 to 0.745, v1 from 0.788 to 0.728, and v8 from 0.808 to 0.748.
- v4, whose gradient-boosted regressors interpolate more smoothly, is the most robust: 0.814 to 0.784.
- The losses concentrate where a family's winner is unusual. With moons held out, v3 never learns to send moons-like data to `louvain_knn` and loses 0.70 ARI there; the karate graph costs 0.32.
- Paired against LMM and spectral clustering on the same 140 tasks, no LOFO router differs

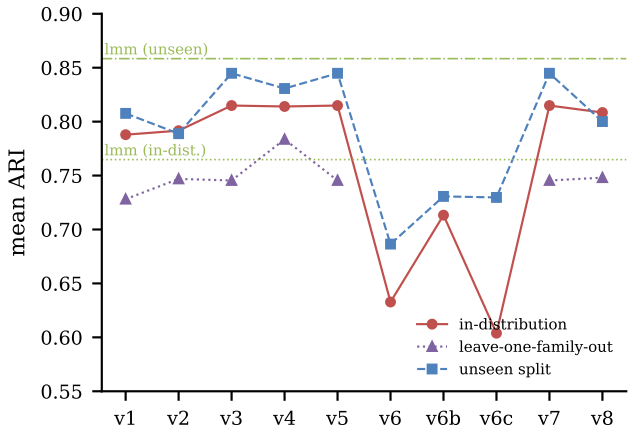


Figure 5: Mean ARI of each router generation in-distribution (task-level leave-one-out), leave-one-family-out (stacked and probe routers v5–v6c not evaluated) and on the OOD split. LMM is shown for reference.

significantly (Holm $p \geq 0.07$; mean differences from -0.037 to $+0.020$).

Unseen configurations. On the OOD split, v3, v5 and v7 (0.845) are level with spectral clustering (0.860), LMM (0.858) and `louvain_knn` (0.846), and behind the four MCG variants (0.890–0.925); none of these differences survives Holm correction. Typical remaining failures are inverse-PCA at d=150 (v3 0.72 against 0.85 for Ward) and outlier extremity 10 (v3 0.74 against 0.92 for `rl_pipeline`). v8's richer fingerprint and OOD-triggered ensemble again did not help (0.800).

An earlier version of this paper reported that v3–v7 lost 0.11–0.13 ARI on the OOD split and that v1 generalised best. That result came from routers trained on a narrower grid (three seeds, no inverse-PCA, synthetic stand-ins for the UCR series) and does not reproduce. The robust conclusion is the LOFO one: a router's advantage comes from having seen a family's winner.

Self-extension (development history, before the controls). Routers retrain from the results file, so adding `louvain_knn` and re-running the benchmark was enough for every fingerprint router to start dispatching to it on the karate graph. This raised the mean ARI of v3, v4, v5 and v7 by 0.029–0.038, the largest router gain since v3, with zero router code changed. Round 15's additions likewise lifted v7 from 0.871 to 0.912. The LOFO result above is the other side of the same mechanism: a router can only dispatch to a winner it has seen.

4.8 Machine-proposed algorithms

The card mutator predicted 0.926 ARI for `mutant_kmeans_meta`; it measured 0.747 at introduction and 0.719 in the controlled benchmark (rank 18). It beats its parent `kmeans_trimmed` (0.654) by 0.065, so the hybrid is a real improvement, but the predicted magnitude was not. Bias matching can order architectures but cannot calibrate them.

Of the round-14 proposals, the two adversarial datasets did what they were designed to do: routers trailed non-routers by 0.036 and 0.068 ARI on them. Of the three new algorithms, none reached the top 20.

Trident (rank 17, 0.725) shows the limit of silhouette gating. On moons it scored 0.262 even though its own `louvain_knn` probe scored 0.997, because a compact wrong split can have a higher silhouette than a correct non-convex one.

4.9 `rl_pipeline`

The behaviour-cloned policy reached 0.59 validation accuracy at predicting the next action. Chance is 0.125 and the majority-class baseline is 0.41. In the original 300-point smoke test it scored ARI 1.00 on blobs and 1.00 on circles, where k-means scores -0.003. On circles it chose `eigen_embed` early: from imitation alone, it learned to switch to the spectral embedding when the cost stops falling. On moons it scored 0.26, because a k-means-like rollout won the silhouette tie-break.

Reproducibility. The traces and checkpoint lived in a git-ignored directory, which is why the agent produced no rows in round 14. The agent now rebuilds both deterministically when they are missing. The rebuilt traces match the originals exactly: 419 rows, 50 episodes, and the same action counts. A checkpoint retrained on them (PyTorch 2.14) reproduces blobs (1.00) and moons (0.30), but scores *0.16* on the circles smoke test, not 1.00. That headline result therefore depended on one lucky training run.

Controlled rerun. The agent ranks 46th of 61 in-distribution (mean ARI 0.604) and is the slowest method (median 2.4 s per task), with a bimodal profile: 0.989 on the SBM graph and 0.907 on anisotropic blobs, but -0.001 on circles and 0.27 on moons at n=500. It does better OOD (rank 23, 0.764), where it is the best algorithm on outlier extremity 10 (0.92).

- At n=500 the policy still fires `eigen_embed`, but every rollout then stalls on `update_centers` no-ops. 419 imitation rows cannot teach the coordinated sequence that completes a spectral partition.
- This is the behaviour-cloning ceiling. Imitation of per-step decisions does not value a move by where the episode ends, and compounding errors off the demonstrator distribution are expected [46].

5 Lessons

1. **Targeted fixes often open new failure modes.** Two of the five one-line fixes (`dbscan_auto`, `clarans_pp`) and two of the three pipeline v2s (AURA, RAPID) traded the diagnosed failure for a new one. The clean pipeline win was META v2, where a brittle rule was strictly replaced.
2. **Dispatch pays only when versions are complementary on detectable regimes.** AURA-v3 (+0.142) succeeded because effective rank separates moons from circles. RAPID-v3 could not help because v2 never won. META-v3 lost because its probe was noisier than the v2 it refined.
3. **Stacking needs architectural diversity.** Re-fitting the same features with a different model (v4) added nothing. A different signal (v6's probe) added a real, if small, gain.
4. **Learned selection does not transfer to unseen families.** Routers beat every single algorithm in-distribution, but with the test family held out they are statistically tied with LMM and spectral clustering. Only a family-level split shows this; a seed-level or configuration-level hold-out does not. On real datasets missing from their training table they tie the best single methods (0.30), and on curved shapes they pick centroid methods (0.31–0.35). Outside the benchmark's families, graph-based methods (the MCG family, LMM, spectral clustering, `louvain_knn`) are as good a default and much cheaper.
5. **Grow the registry, not the router.** The biggest router gains came from adding algorithms and data, not from router redesigns.
6. **Internal-validity gates are shape-biased.** Silhouette-based probes (v6, META-v3, Trident) repeatedly pick compact wrong answers on non-convex data. A connectivity-aware criterion or shape prior is needed.

7. **Imitation recovers moves, not plans.** The RL agent learned to fire `eigen_embed` as a conditional move but not the sequence that makes it pay off, and its best smoke result depended on one training run.
8. **Silent failures bias means.** GMM's singular covariances, a YAML float parsed as a string, a missing default argument and git-ignored training files each removed rows from the benchmark without stopping it. Each missing row changed the means it was dropped from until it was root-caused.
9. **The graph, not the point count, is what graph methods need.** A thousand micro-clusters linked by the points that lie between them carry the connectivity signal that makes LMM and spectral clustering accurate, at linear cost. Profile before optimising: the first speed-up came from the neighbour search, not the model.
10. **Edge weights trade cuts for geometry.** Gaussian weights that separate close rings and outliers distort the eigenvectors of smooth, gap-free manifolds, and plain boundary counts do the opposite. Giving EM both bases was the best compromise we found, and an eigengap switch between them was not.
11. **State the size of the comparison family.** We first reported an OOD win of `lmm_mcg` over LMM as significant after Holm correction over three comparisons. The candidates were four variants against three references, and over those twelve tests no OOD win survives. Correcting over the comparisons actually made is part of the result.

Limitations. Most tasks are small ($n \leq 1000$). The big-data sweep reaches $n = 200{,}000$ on one generator with one seed, and the large-n graph comparison uses eleven families at $d = 20$. Twenty families give wide intervals, so many adjacent ranks are ties. Lineage tables come from earlier, smaller grids run before the validity controls. Algorithms run with fixed hyperparameters. Many ideas here have prior art:

- algorithm selection [47];
- meta-learning for clustering, from early work to recent systems [48–52];
- landmarking [53], which our probe features are;
- stacked generalisation [24];
- Laplacian mixture modelling [8];
- exponential-family mixtures [9];
- spectral clustering on centroids and anchor graphs [37, 40, 41], competitive Hebbian topology graphs [35, 36], locally scaled affinities [39] and BIRCH's CF triples [38], which MCG combines.

Recent clustering-recommendation systems such as ML2DAC, CLAMS and ClustRecNet target the same problem as our routers and should be compared against directly in future work. A limited search found no published MRI-relaxation clustering method and no prior agent that composes clustering algorithms from primitive operators learned by imitation, but neither search was exhaustive. Our contribution is a systematic, re-runnable evaluation of these ideas against a broad registry, with validity controls, plus the FMM/LMM/AMM variants, the trajectory-trained agent and the MCG combination.

6 Conclusion and future work

Starting from benchmark failure modes, we built 39 algorithms and ten learned selectors and evaluated all of them against one another, including on data none had seen. The most durable results are these:

- graph-based clustering by connectivity, in the form of LMM, an EM-trained mixture over a graph-Laplacian basis;
- its linear-cost counterpart MCG, which keeps that accuracy on a graph of micro-clusters and is 54 times faster at $n = 100{,}000$;

- an explanation of why both work on manifolds: their eigenvectors are Fourier modes along the data;
- a negative result: learned routers win only on dataset families they have seen.

Four directions follow:

- fine-tune the behaviour-cloned agent with PPO [54], rewarding terminal partition quality, using the trajectory corpus every benchmark run now produces;
- replace specific weak components, such as CLARANS's swap proposal and Chameleon's merge rule, with models trained on trajectory data;
- give selection gates a shape-aware criterion, e.g. penalise silhouette when the number of kNN-graph components disagrees with k;
- adapt MCG's resolution to local structure, so that tightly packed 2-D arms get enough micro-clusters, and replace LMM's (k, M, n) Newton step so that the EM-based variants scale in k.

Acknowledgement of AI assistance. The benchmark harness's initial scaffold was written by the author. The algorithm implementations, experiments, analysis scripts and drafts of this paper were produced with an AI coding assistant (Claude Code), directed and reviewed by the author, who takes responsibility for the content.

Availability. All algorithms, configurations, results and the figure scripts are at `https://github.com/Ashail33/clustbench`.

References

[1] Ashail Maharaj. Clustbench: A reproducible, trajectory-instrumented, self-extending benchmark for clustering algorithms, 2026. Companion paper, `https://github.com/Ashail33/clustbench/tree/master/paper/clustbench`.

[2] Jon Kleinberg. An impossibility theorem for clustering. In *Advances in Neural Information Processing Systems*, volume 15, 2002.

[3] David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, 1997.

[4] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.

[5] A. P. Dempster, N. M. Laird, and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B*, 39(1):1–38, 1977.

[6] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, volume 20, 2007.

[7] Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17(4): 395–416, 2007.

[8] Daniel Korenblum. Laplacian mixture modeling for network analysis and unsupervised learning on graphs. *PLOS ONE*, 13(10):e0204096, 2018. arXiv:1502.00727.

[9] Arindam Banerjee, Srujana Merugu, Inderjit S. Dhillon, and Joydeep Ghosh. Clustering with Bregman divergences. *Journal of Machine Learning Research*, 6:1705–1749, 2005.

[10] Bharath Sriperumbudur, Kenji Fukumizu, Arthur Gretton, Aapo Hyvärinen, and Revant Kumar. Density estimation in infinite dimensional exponential families. *Journal of Machine Learning Research*, 18(57):1–59, 2017.

[11] Radha Chitta, Rong Jin, and Anil K. Jain. Efficient kernel clustering using random Fourier features. In *2012 IEEE 12th International Conference on Data Mining*, 2012.

[12] Kamal Nigam and Rayid Ghani. Analyzing the effectiveness and applicability of co-training. In *Proceedings of the Ninth International Conference on Information and Knowledge Management*, pages 86–93, 2000.

[13] Steffen Bickel and Tobias Scheffer. Multi-view clustering. In *Proceedings of the Fourth IEEE International Conference on Data Mining*, pages 19–26, 2004.

[14] Martin Ester, Hans-Peter Kriegel, Jörg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.

[15] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[16] Rasim M. Alguliyev, Ramiz M. Aliguliyev, and Lyudmila V. Sukhostat. Weighted consensus clustering and its application to big data. *Expert Systems with Applications*, 150:113294, 2020.

[17] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. LOF: Identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 93–104, 2000.

[18] Christopher K. I. Williams and Matthias Seeger. Using the Nyström method to speed up kernel machines. In *Advances in Neural Information Processing Systems*, volume 13, 2001.

[19] Charless Fowlkes, Serge Belongie, Fan Chung, and Jitendra Malik. Spectral grouping using the Nyström method. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26 (2):214–225, 2004.

[20] M. E. J. Newman. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23):8577–8582, 2006.

[21] Wayne W. Zachary. An information flow model for conflict and fission in small groups. *Journal of Anthropological Research*, 33(4):452–473, 1977.

[22] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008(10):P10008, 2008.

[23] Elizaveta Levina and Peter J. Bickel. Maximum likelihood estimation of intrinsic dimension. In *Advances in Neural Information Processing Systems*, volume 17, 2005.

[24] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2):241–259, 1992.

[25] Alex Rodriguez and Alessandro Laio. Clustering by fast search and find of density peaks. *Science*, 344(6191):1492–1496, 2014.

[26] Dengyong Zhou, Olivier Bousquet, Thomas Navin Lal, Jason Weston, and Bernhard Schölkopf. Learning with local and global consistency. In *Advances in Neural Information Processing Systems*, volume 16, 2004.

[27] Hendrik Fichtenberger, Marc Gillé, Melanie Schmidt, Chris Schwiegelshohn, and Christian Sohler. BICO: BIRCH meets coresets for k-means clustering. In *Algorithms – ESA 2013*, volume 8125 of *Lecture Notes in Computer Science*, pages 481–492, 2013.

[28] Sariel Har-Peled and Soham Mazumdar. On coresets for k-means and k-median clustering. In *Proceedings of the 36th Annual ACM Symposium on Theory of Computing*, pages 291–300, 2004.

[29] Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for nonlinear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000.

[30] David M. Blei and Michael I. Jordan. Variational inference for Dirichlet process mixtures. *Bayesian Analysis*, 1(1):121–143, 2006.

[31] Dean A. Pomerleau. ALVINN: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, volume 1, 1989.

[32] Wei Dong, Moses Charikar, and Kai Li. Efficient k-nearest neighbor graph construction for generic similarity measures. In *Proceedings of the 20th International Conference on World Wide Web*, pages 577–586, 2011.

[33] Andrew V. Knyazev. Toward the optimal preconditioned eigensolver: Locally optimal block preconditioned conjugate gradient method. *SIAM Journal on Scientific Computing*, 23(2): 517–541, 2001.

[34] Nathan Bell, Luke N. Olson, and Jacob Schroder. PyAMG: Algebraic multigrid solvers in Python. *Journal of Open Source Software*, 7(72):4142, 2022.

[35] Thomas Martinetz and Klaus Schulten. Topology representing networks. *Neural Networks*, 7(3):507–522, 1994.

[36] Kadim Taşdemir and Erzsébet Merényi. Exploiting data topology in visualization and clustering of self-organizing maps. *IEEE Transactions on Neural Networks*, 20(4):549–562, 2009.

[37] Donghui Yan, Ling Huang, and Michael I. Jordan. Fast approximate spectral clustering. In *Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 907–916, 2009.

[38] Tian Zhang, Raghu Ramakrishnan, and Miron Livny. BIRCH: An efficient data clustering method for very large databases. In *Proceedings of the 1996 ACM SIGMOD International Conference on Management of Data*, pages 103–114, 1996.

[39] Lihi Zelnik-Manor and Pietro Perona. Self-tuning spectral clustering. In *Advances in Neural Information Processing Systems*, volume 17, 2004.

[40] Wei Liu, Junfeng He, and Shih-Fu Chang. Large graph construction for scalable semi-supervised learning. In *Proceedings of the 27th International Conference on Machine Learning*, pages 679–686, 2010.

[41] Dong Huang, Chang-Dong Wang, Jian-Sheng Wu, Jian-Huang Lai, and Chee-Keong Kwoh. Ultra-scalable spectral clustering and ensemble clustering. *IEEE Transactions on Knowledge and Data Engineering*, 32(6):1212–1226, 2020.

[42] Asa Ben-Hur, David Horn, Hava T. Siegelmann, and Vladimir Vapnik. Support vector clustering. *Journal of Machine Learning Research*, 2:125–137, 2001.

[43] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[44] Mikhail Belkin and Partha Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003.

[45] Ronald R. Coifman and Stéphane Lafon. Diffusion maps. *Applied and Computational Harmonic Analysis*, 21(1):5–30, 2006.

[46] Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

[47] John R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

[48] Marcilio C. P. de Souto, Ricardo B. C. Prudêncio, Rodrigo G. F. Soares, Daniel S. A. de Araujo, Ivan G. Costa, Teresa B. Ludermir, and Alexander Schliep. Ranking and selecting clustering algorithms using a meta-learning approach. In *2008 IEEE International Joint Conference on Neural Networks*, pages 3729–3735, 2008.

[49] Daniel G. Ferrari and Leandro Nunes de Castro. Clustering algorithm selection by meta-learning systems: A new distance-based problem characterization and ranking combination methods. *Information Sciences*, 301:181–194, 2015.

[50] Dennis Treder-Tschechlov, Manuel Fritz, Holger Schwarz, and Bernhard Mitschang. ML2DAC: Meta-learning to democratize AutoML for clustering analysis. *Proceedings of the ACM on Management of Data*, 1(2), 2023. SIGMOD 2023, doi:10.1145/3589289.

[51] Prabhant Singh, Pieter Gijsbers, Murat Onur Yildirim, Elif Ceren Gok, and Joaquin Vanschoren. CLAMS: A system for zero-shot model selection for clustering. *arXiv preprint arXiv:2407.11286*, 2024.

[52] Mohammadreza Bakhtyari, Bogdan Mazoure, Renato Cordeiro de Amorim, Guillaume Rabusseau, and Vladimir Makarenkov. ClustRecNet: A novel end-to-end deep learning framework for clustering algorithm recommendation. *arXiv preprint arXiv:2509.25289*, 2025.

[53] Bernhard Pfahringer, Hilan Bensusan, and Christophe Giraud-Carrier. Meta-learning by landmarking various learning algorithms. In *Proceedings of the 17th International Conference on Machine Learning*, pages 743–750, 2000.

[54] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Table 4: Controlled benchmark. Mean ARI with rank by mean ARI in-distribution (61 algorithms) and OOD (62 algorithms); LOFO is the routers' mean ARI when retrained without the family under test. Median wall time (`fit_predict` only) is in-distribution. Groups: basis mixtures, micro-cluster graph methods, other new algorithms, routers, classical baselines.

algorithm	ARI (in)	rank	LOFO	ARI (OOD)	rank	wall (s)
`lmm`	0.765	9	–	0.858	8	0.033
`fmm`	0.657	34	–	0.728	34	0.152
`amm`	0.625	44	–	0.716	41	0.482
`tbmm`	0.760	13	–	0.869	5	0.345
`tbmm_fl`	0.759	14	–	0.867	6	0.065
`flhmm`	0.652	39	–	0.724	37	0.064
`flh_cycle_product`	0.627	43	–	0.676	45	0.100
`flh_cycle`	0.381	57	–	0.344	59	0.102
`mcg`	0.744	15	–	0.890	4	0.012
`mcg_cf`	0.761	12	–	0.917	2	0.021
`lmm_mcg`	0.764	10	–	**0.925**	**1**	0.023
`lmm_mcg_dual`	0.769	8	–	0.892	3	0.044
`mri`	0.562	50	–	0.617	52	0.032
`louvain_knn`	0.714	19	–	0.846	9	0.098
`aura_v3`	0.737	16	–	0.831	14	0.024
`aura_v2`	0.714	20	–	0.832	13	0.040
`rapid`	0.702	22	–	0.727	35	0.051
`rapid_v3`	0.699	27	–	0.726	36	0.065
`meta_clusterer_v2`	0.701	24	–	0.628	51	0.084
`pwcc_diverse`	0.701	23	–	0.756	25	0.069
`mutant_kmeans_meta`	0.719	18	–	0.756	24	0.058
`trident`	0.725	17	–	0.826	16	0.148
`bico_stream`	0.677	31	–	0.719	39	0.030
`isomap_bgmm`	0.634	41	–	0.766	22	0.160
`density_peaks_bridge`	0.546	51	–	0.708	42	0.035
`rl_pipeline`	0.604	46	–	0.764	23	2.434
`learned_router` (v1)	0.788	7	0.728	0.808	17	0.203
`learned_router_v2`	0.792	6	0.747	0.789	19	0.144
`learned_router_v3`	**0.815**	**1**	0.745	0.845	12	0.109
`learned_router_v4`	0.814	4	**0.784**	0.831	15	0.235
`learned_router_v7`	0.815	3	0.745	0.845	11	0.167
`learned_router_v8`	0.808	5	0.748	0.800	18	0.103
`spectral`	0.763	11	–	0.860	7	0.027
`gmm`	0.699	26	–	0.774	21	0.014
`agglomerative` (Ward)	0.685	29	–	0.754	26	0.006
`kmeans`	0.655	36	–	0.731	29	0.006